Intelligent Systems,
MIPT

Natural Language Processing

L7 / 31.03.26

- Course page
- GitHub

Content

Lecture 7: Training Large Language Models

- Training on a single GPU
- Mixed-Precision Training: BF16, FP32, FP8
- Data Parallelism and All-Reduce
- ZeRO and Fully Sharded Data Parallel (FSDP)

Motivation

- **The scale of modern training:**

  > **Llama 3 405B:** 16,000 H100 GPUs & batch size ~4M tokens & 15T tokens of data

  > **DeepSeek-V3:** 2,048 H800 GPUs & batch size up to 60M tokens & 14.8T tokens

- **Key challenges when scaling:**

Memory

If a training step doesn't fit in memory, training simply cannot proceed

Compute

GPUs should compute, not sit idle. We must minimize idle time

Communication

Data transfers between GPUs are expensive. Need to overlap them with compute

Training on One GPU

- **Basics of model training:**

  > Forward pass
  > Backward pass
  > Optimization

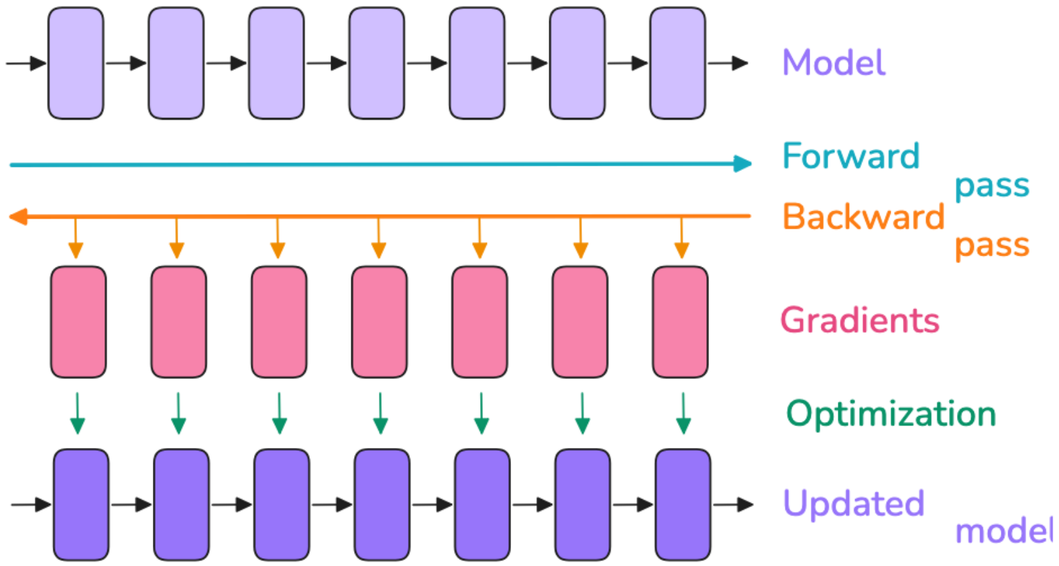
- **What lives in GPU memory during training:**

  > Model weights (parameters)

  > Gradients

  > Optimizer state

  > Activations



Training on One GPU: details

- **What lives in GPU memory during training*:**

  > Model weights (parameters)

$$N = h * v + L * (12 * h^2 + 13 * h) + 2 * h$$

where h – hidden dimension, v – vocabulary size, and L – number of layers in the model

$$m_{params} = 4 * N$$

* fp32 case

Training on One GPU: details

- **What lives in GPU memory during training*:**

  > Model weights (parameters)

$$N = h * v + L * (12 * h^2 + 13 * h) + 2 * h$$

where h – hidden dimension, v – vocabulary size, and L – number of layers in the model

$$m_{params} = 4 * N$$

  > Gradients

$$m_{grad} = 4 * N$$

* fp32 case

Training on One GPU: details

- **What lives in GPU memory during training*:**

  > Model weights (parameters)

  $$N = h * v + L * (12 * h^2 + 13 * h) + 2 * h$$

  where h – hidden dimension, v – vocabulary size, and L – number of layers in the model

  $$m_{params} = 4 * N$$

  > Gradients

  $$m_{grad} = 4 * N$$

  > Optimizer state

  $$m_{opt} = (4 + 4) * N$$

* fp32 case

Training on One GPU: details

- **What lives in GPU memory during training*:**

  > Model weights (parameters)

  $$N = h * v + L * (12 * h^2 + 13 * h) + 2 * h$$

  where h – hidden dimension, v – vocabulary size, and L – number of layers in the model

  $$m_{params} = 4 * N$$

  > Gradients

  $$m_{grad} = 4 * N$$

  > Optimizer state

  $$m_{opt} = (4 + 4) * N$$

  > Activations

  $$m_{act} = L \cdot seq \cdot bs \cdot h \cdot (34 + \frac{5 \cdot n_{heads} \cdot seq}{h})$$

  where seq – sequence length, bs – batch size in samples, n_{heads} – number of heads.

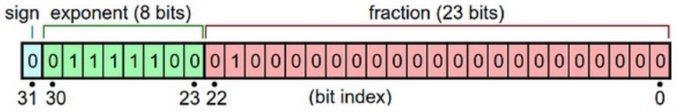
* fp32 case

Mixed Precision

- **The scale of modern training:**

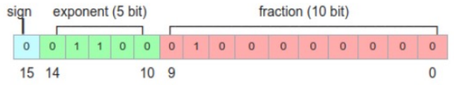
  > **Sign:** the first bit determines if the number is positive or negative
  > **Mantissa:** determines the significant figures of a number
  > **Exponent:** controls the magnitude of the number

FP32



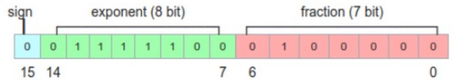
$from\ -10^{38}\ up\ to\ 10^{38}$

FP16



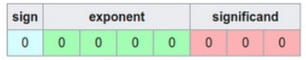
$from\ -65504\ \ up\ to\ 65504$

BF16



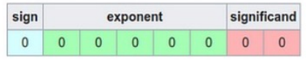
$from\ -10^{38}\ up\ to\ 10^{38}$

FP8 (E4M3)



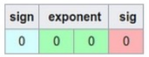
$from\ -448\ up\ to\ 448$

FP8 (E5M2)



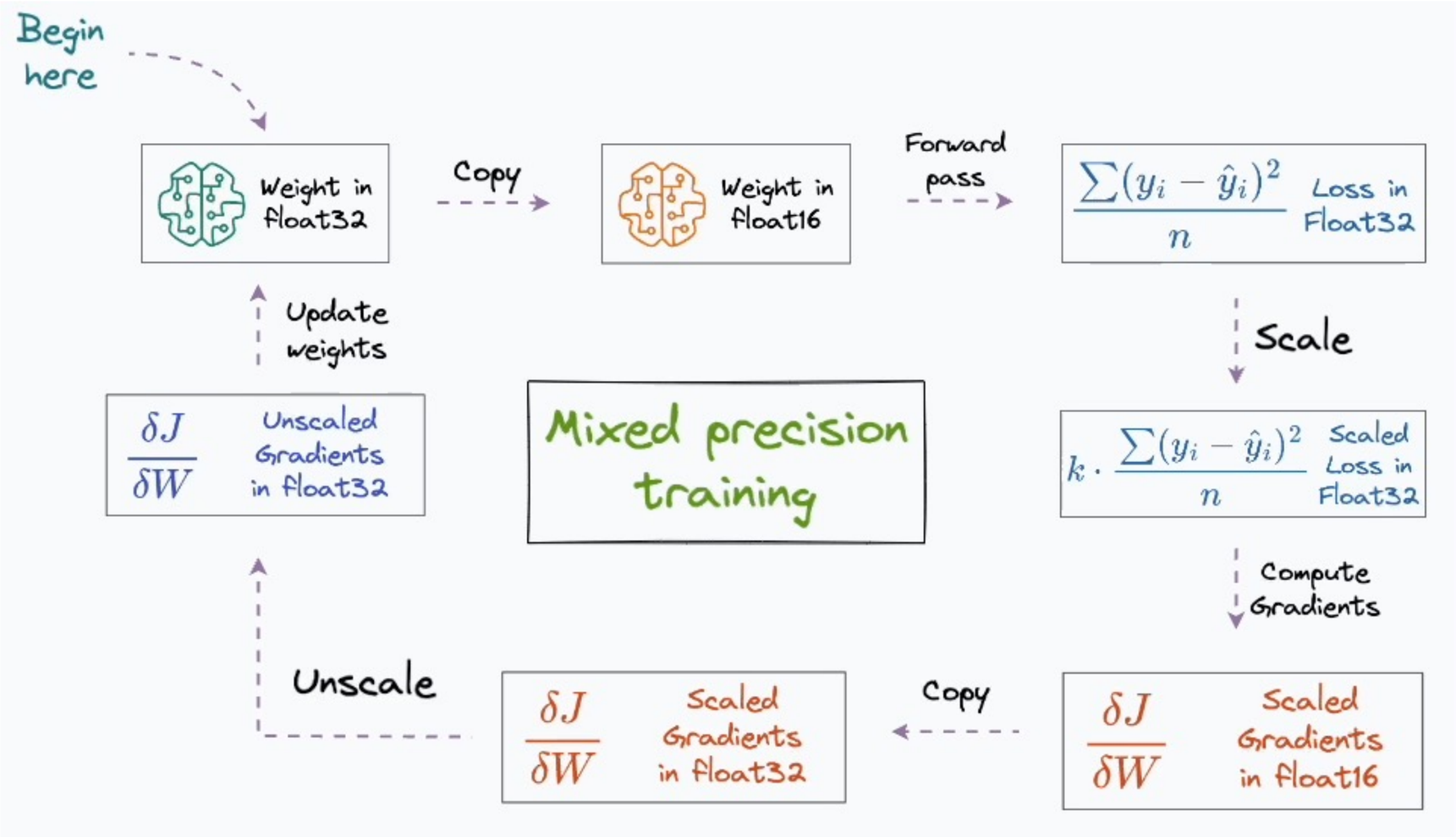
$from\ -57344\ up\ to\ 57344$

FP4 (E2M1)



$from\ -8\ up\ to\ 8$

Mixed Precision Training



Training on One GPU: details

- **What lives in GPU memory during training*:**

> Model weights (parameters)

$$N = h * v + L * (12 * h^2 + 13 * h) + 2 * h$$

where h – hidden dimension, v – vocabulary size, and L – number of layers in the model

$$\cancel{m_{params} = 4 * N} \qquad m_{params} = 2 * N \quad + \quad m_{params_fp32} = 4 * N$$

> Gradients

$$\cancel{m_{grad} = 4 * N} \qquad m_{grad} = 2 * N$$

> Optimizer state

$$\cancel{m_{opt} = (4 + 4) * N} \qquad m_{opt} = (4 + 4) * N$$

> Activations

$$m_{act} = L \cdot seq \cdot bs \cdot h \cdot (34 + \frac{5 \cdot n_{heads} \cdot seq}{h})$$

where seq – sequence length, bs – batch size in samples, n_{heads} – number of heads.

* mixed precision case

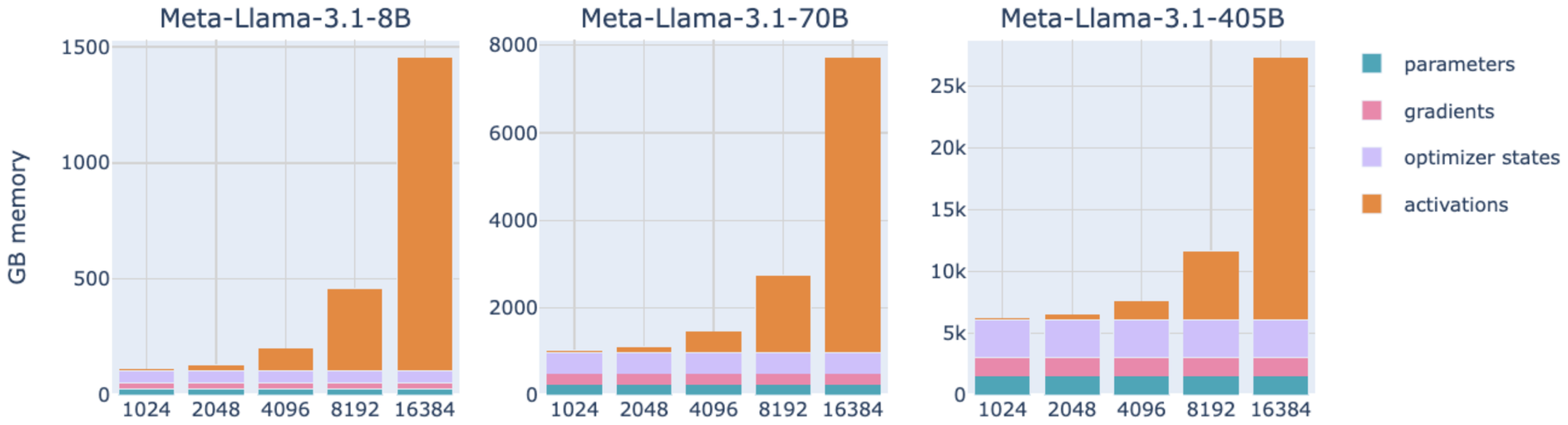
Example of memory usage

Parameters	FP32 / BF16 mixed	Fits on H100 (80 GB)?
1B	16–20 GB	✓ fits
7B	112–140 GB	✗ doesn't fit
70B	1,120–1,400 GB	✗✗ needs 14–18 GPUs
405B	6,480–8,100 GB	✗✗✗ needs 81–101 GPUs

Example of memory usage

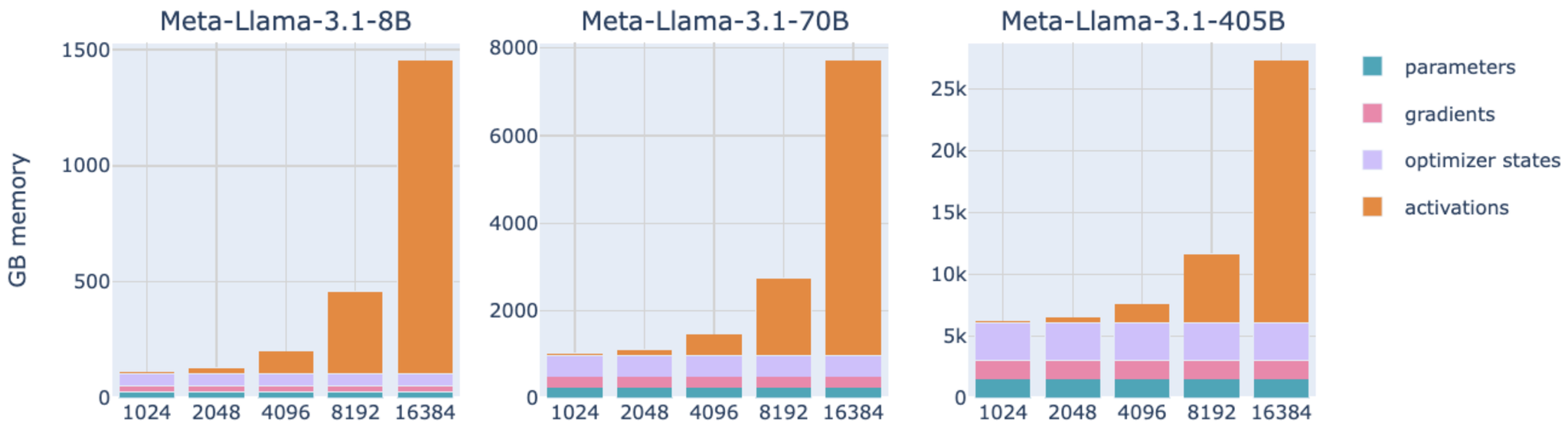
Parameters	FP32 / BF16 mixed	Fits on H100 (80 GB)?
1B	16–20 GB	✓ fits
7B	112–140 GB	✗ doesn't fit
70B	1,120–1,400 GB	✗✗ needs 14–18 GPUs
405B	6,480–8,100 GB	✗✗✗ needs 81–101 GPUs

- **The activation memory is the part which will blow up when we increase our batch size or train with longer sequences**



Example of memory usage

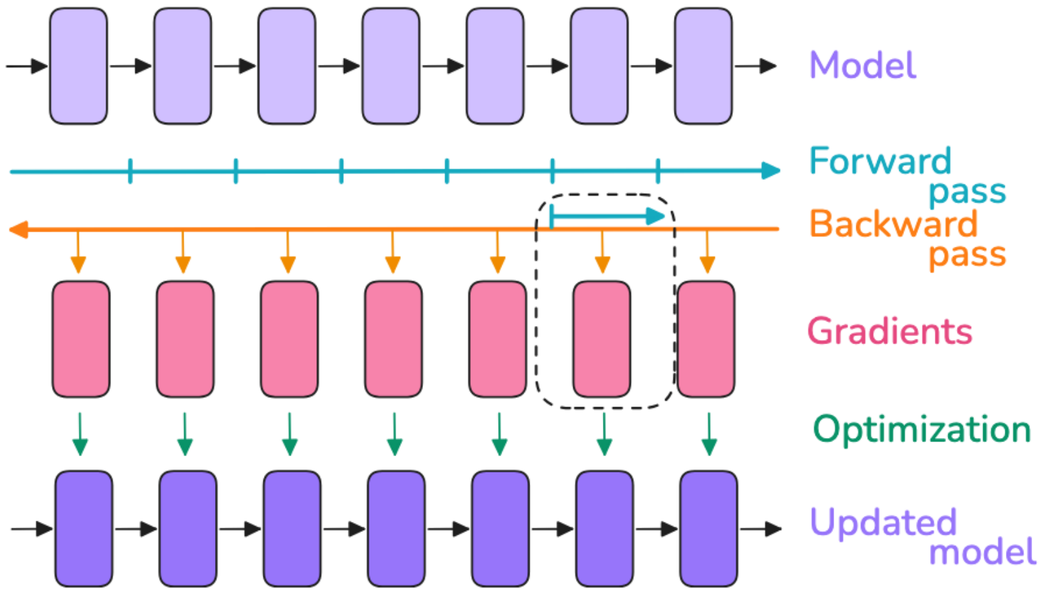
- **The activation memory is the part which will blow up when we increase our batch size or train with longer sequences**



- **Is there a way to tame this "activation explosion"?**

Gradient checkpointing

- **Idea:** discard some activations during the forward pass to save memory and spend some extra compute to recompute these on the fly during the backward pass

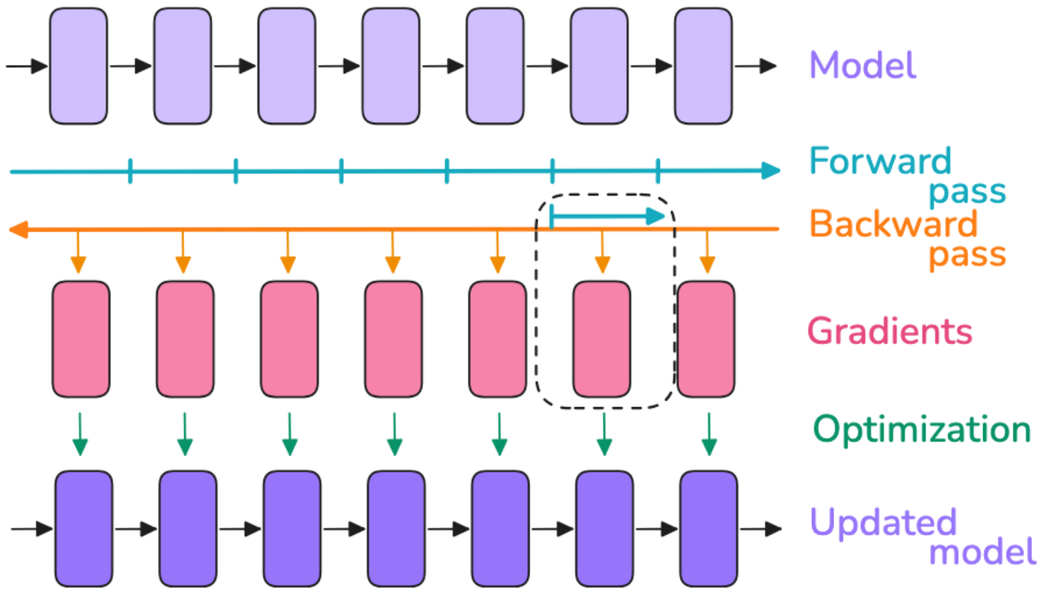


Gradient checkpointing

- **Idea:** discard some activations during the forward pass to save memory and spend some extra compute to recompute these on the fly during the backward pass

> **Full:** saves the most memory, but increases the compute cost and time by up to 30-40%

> **Selective:** for example, for a GPT-3, 70% activation memory reduction at a 2.7% compute cost

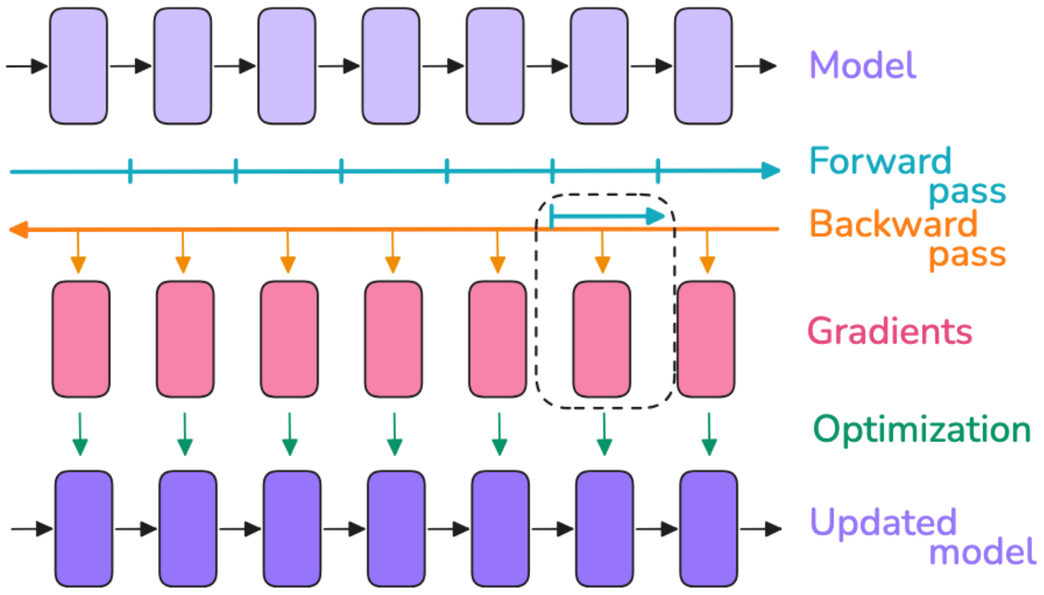


Gradient checkpointing

- **Idea:** discard some activations during the forward pass to save memory and spend some extra compute to recompute these on the fly during the backward pass

> **Full:** saves the most memory, but increases the compute cost and time by up to 30-40%

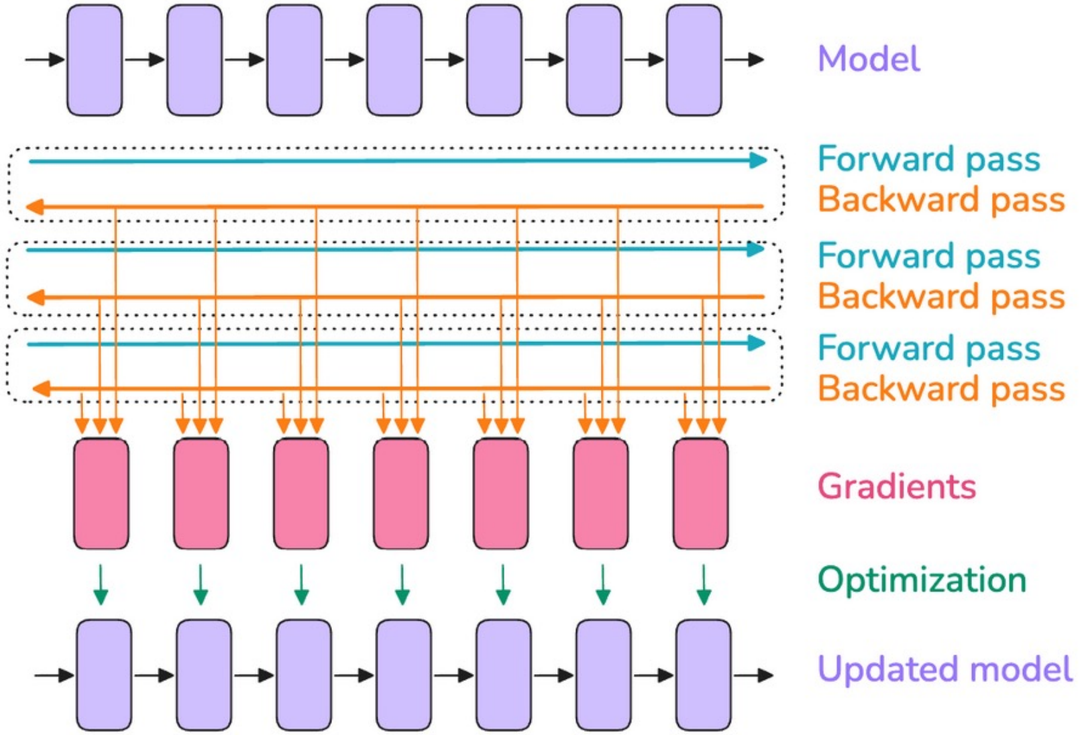
> **Selective:** for example, for a GPT-3, 70% activation memory reduction at a 2.7% compute cost



- Activation recomputation increases the number of FLOPs slightly due to recomputation, while it significantly reduces memory access overhead.

Gradient accumulation

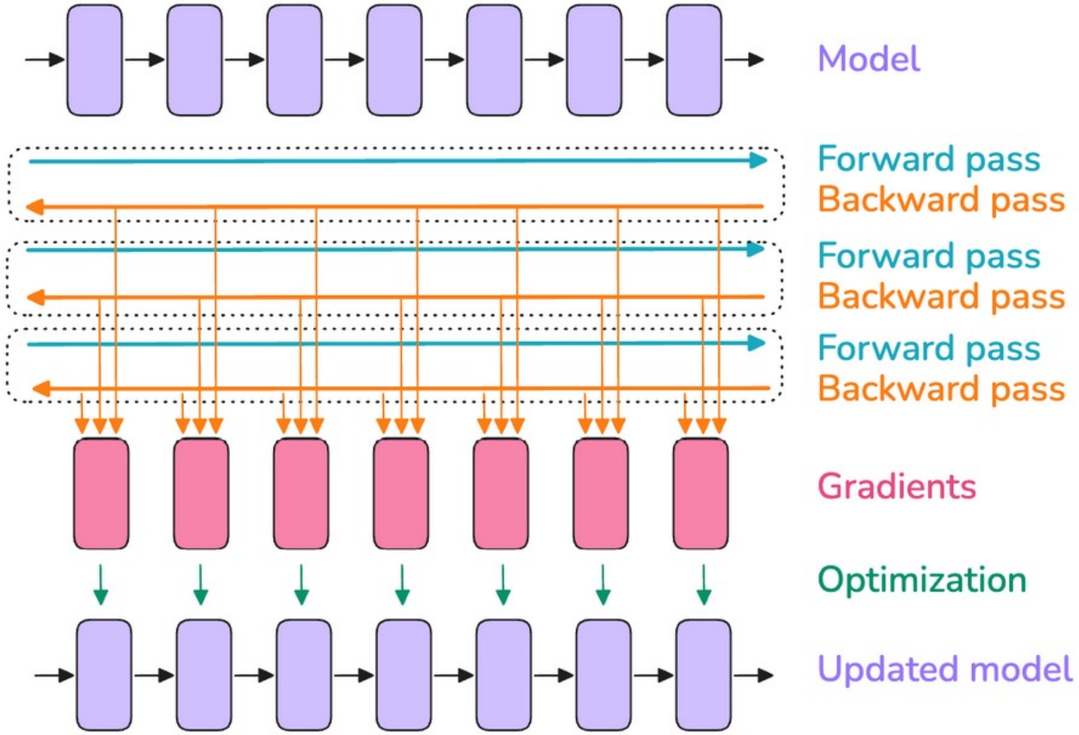
- **Idea:** perform forward and backward passes successively on micro-batches, compute the gradients, and, sum the gradients of all micro-batches before performing an optimizer step.



Gradient accumulation

- **Idea:** **perform forward and backward passes successively on micro-batches, compute the gradients, and, sum the gradients of all micro-batches before performing an optimizer step.**

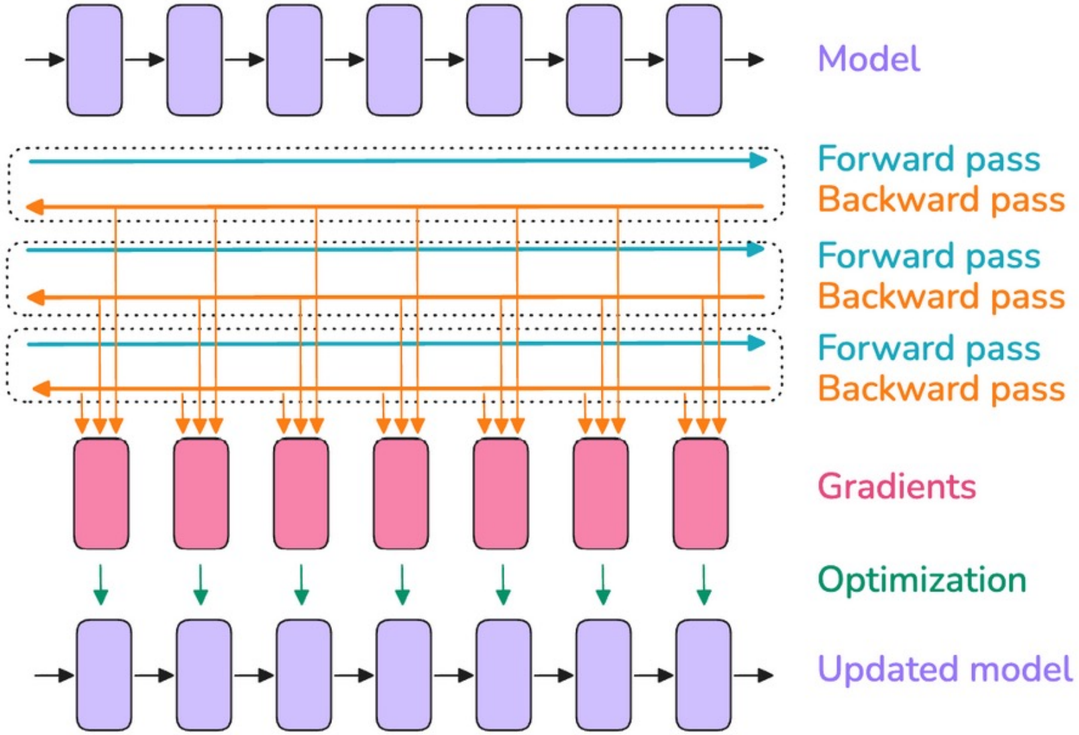
global_batch_size = micro_batch * grad_acum



Gradient accumulation

- **Idea:** perform forward and backward passes successively on micro-batches, compute the gradients, and, sum the gradients of all micro-batches before performing an optimizer step.

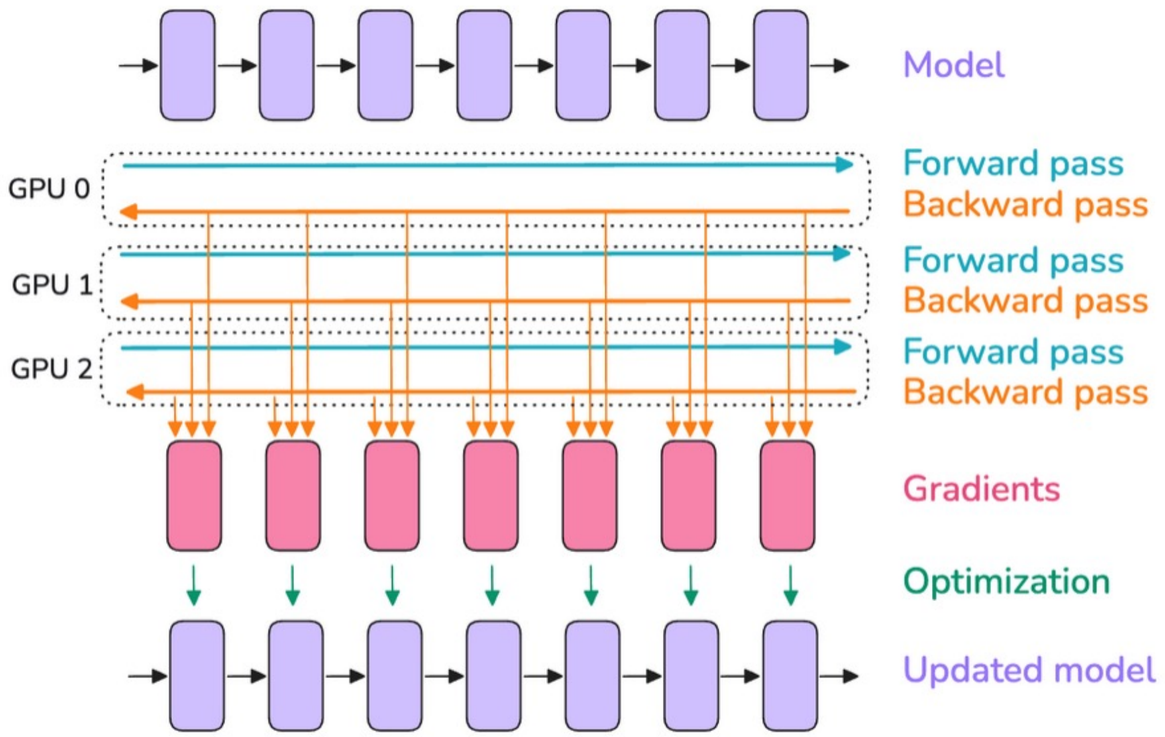
*global_batch_size = micro_batch * grad_acum*



- **Do not forget:** gradient accum requires multiple consecutive forward/backward passes per optimization step thereby increasing the compute overhead and slowing down training

Data Parallelism

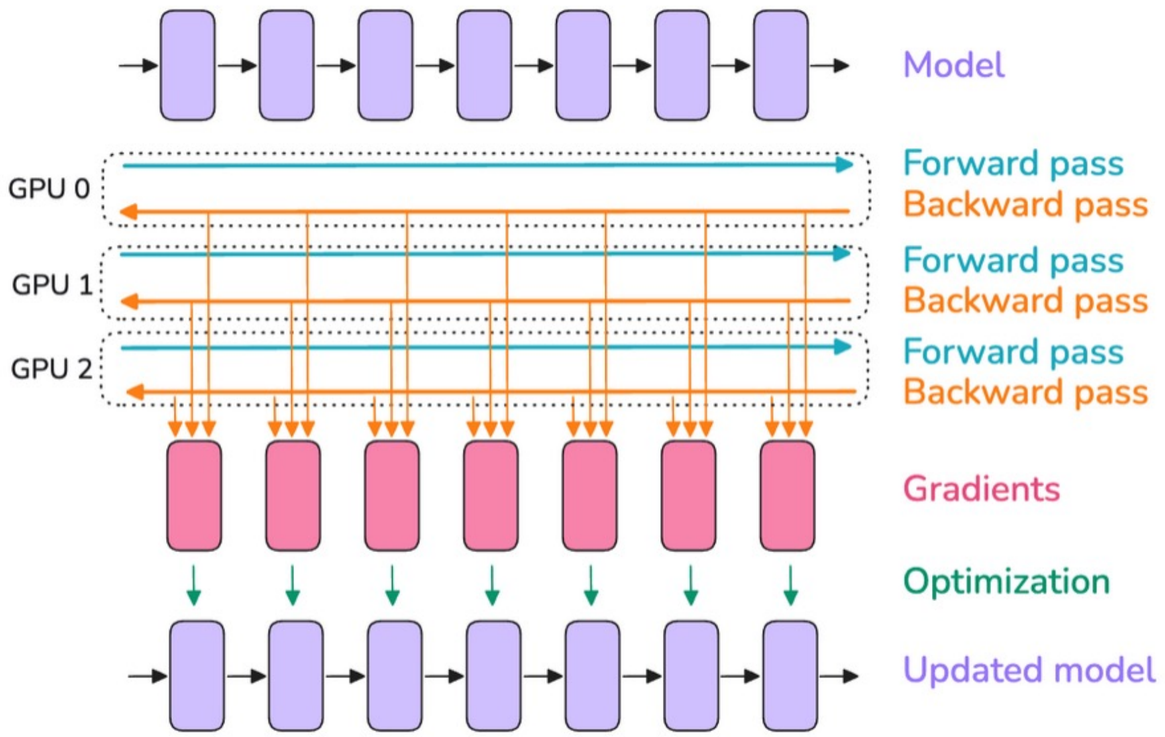
- **Idea:** replicate the model on several GPUs and run forward and backward passes on different micro-batches of data in parallel for each GPU



Data Parallelism

- **Idea: replicate the model on several GPUs and run forward and backward passes on different micro-batches of data in parallel for each GPU**

  gbs = micro_batch * grad_acum * dp

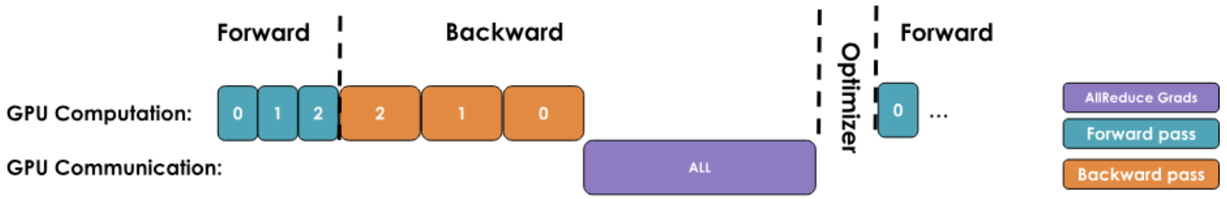


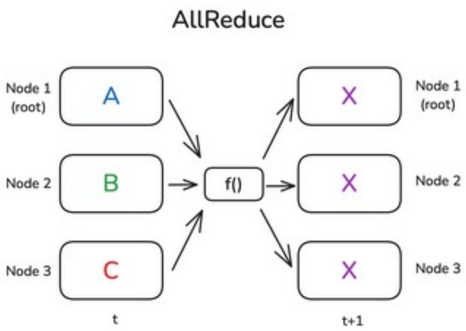
Data Parallelism

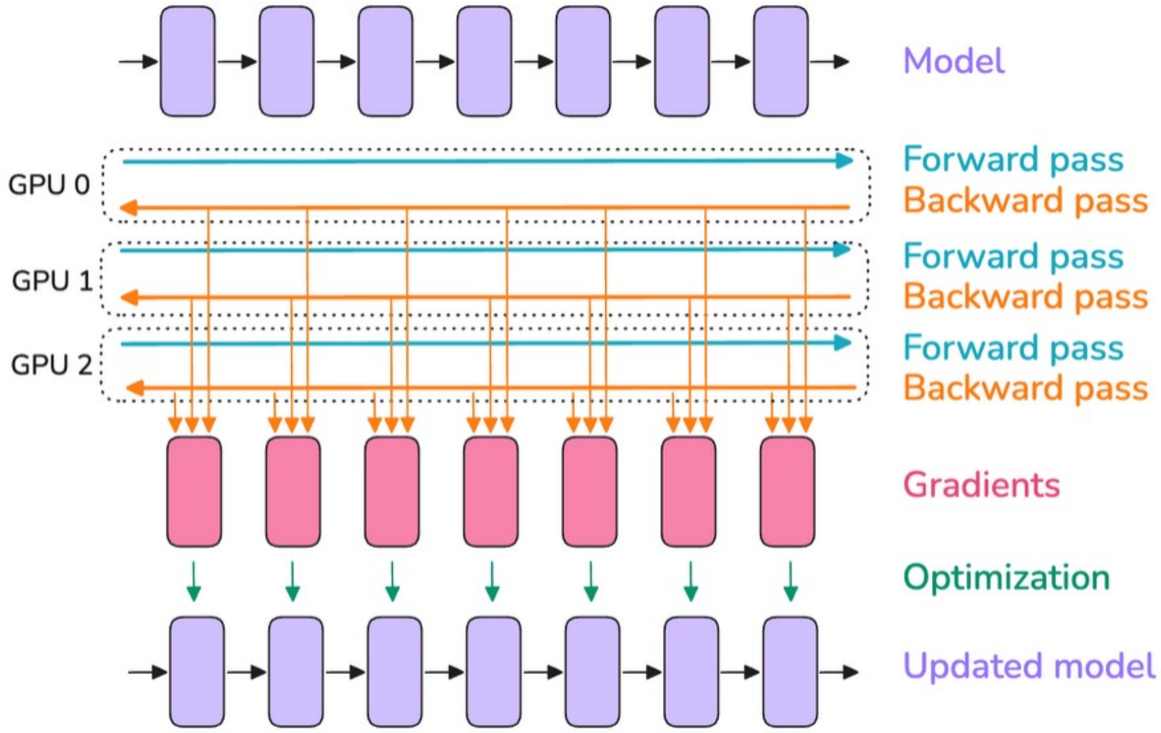
- **Idea:** replicate the model on several GPUs and run forward and backward passes on different micro-batches of data in parallel for each GPU

  *gbs = micro_batch * grad_acum * dp*

- Naïve all-reduce for sync in DDP

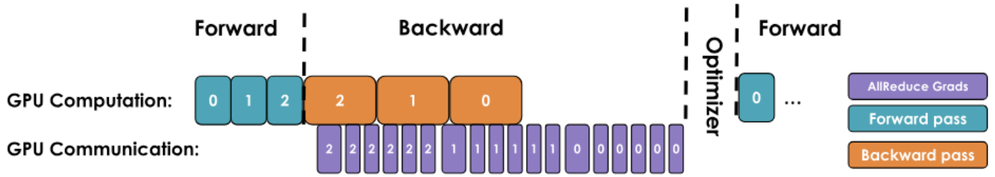






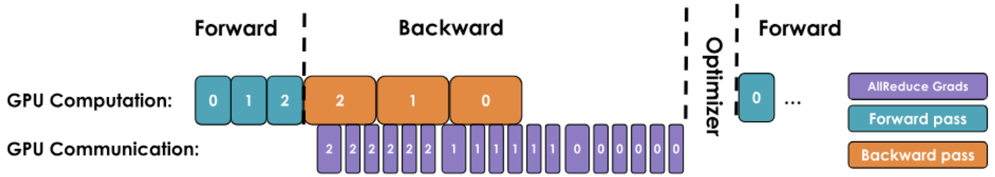
Data Parallelism: optimizations

- **Overlap all-reduce with backward:** as soon as a layer's gradient is ready — launch all-reduce immediately, don't wait

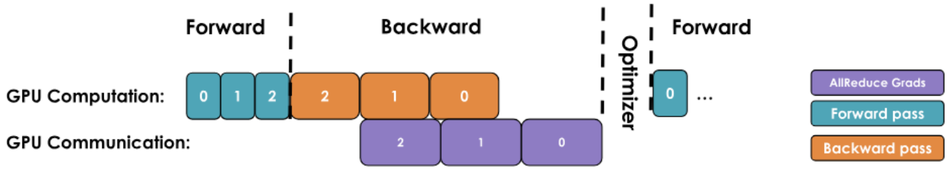


Data Parallelism: optimizations

- **Overlap all-reduce with backward:** as soon as a layer's gradient is ready — launch all-reduce immediately, don't wait

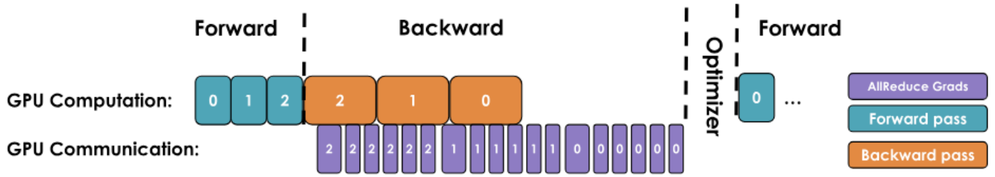


- **Gradient bucketing:** group small gradients into "buckets," one all-reduce per bucket. Larger tensors = more efficient on GPU.

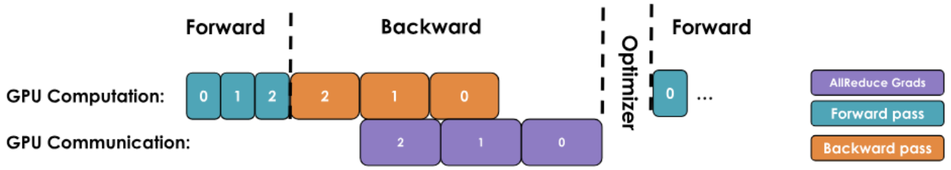


Data Parallelism: optimizations

- **Overlap all-reduce with backward:** as soon as a layer's gradient is ready — launch all-reduce immediately, don't wait



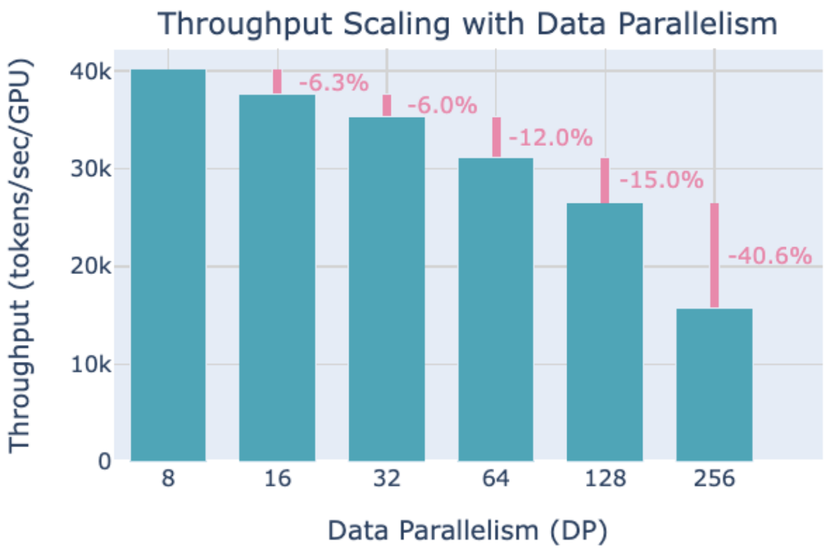
- **Gradient bucketing:** group small gradients into "buckets," one all-reduce per bucket. Larger tensors = more efficient on GPU.

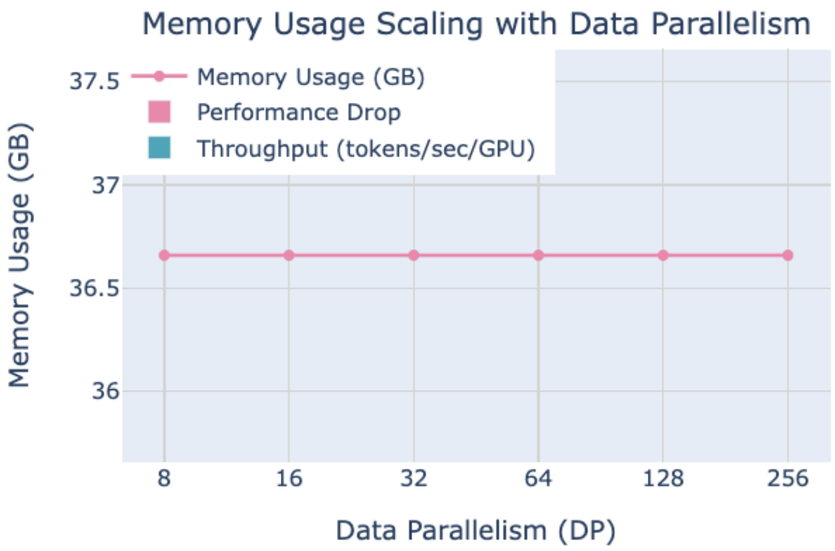


- **no_sync during grad accumulation:** sync only on the last micro-batch. model.no_sync() disables all-reduce on intermediate steps.

Data Parallelism

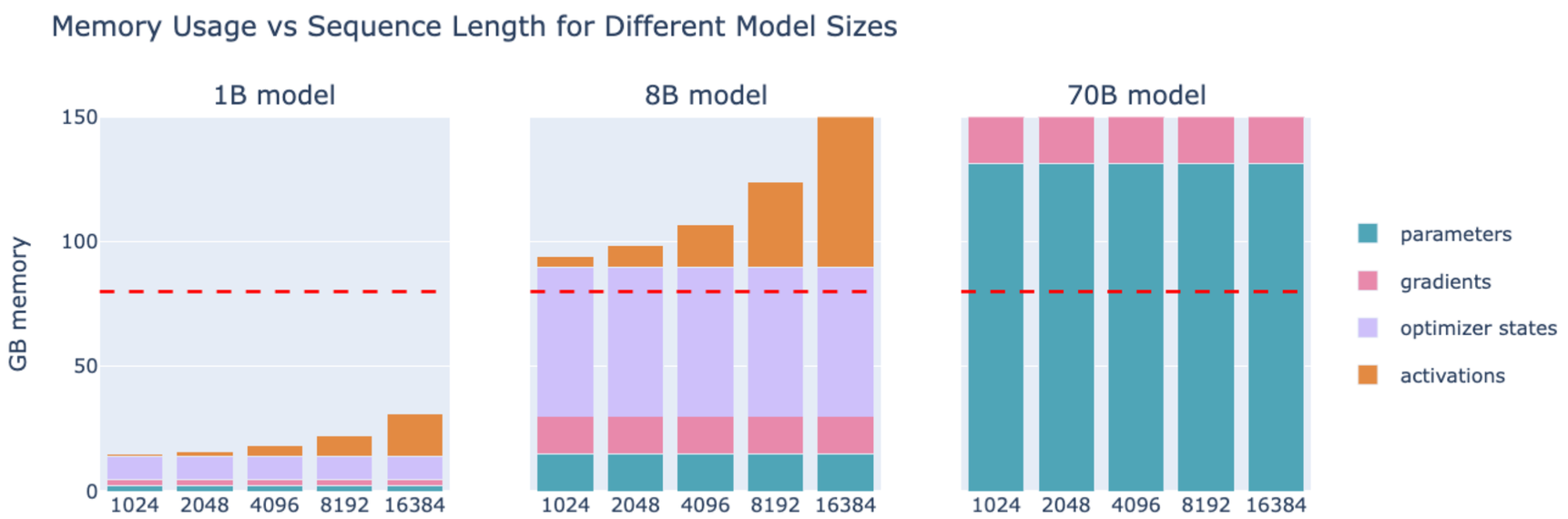
- **In a DP approach, having more GPUs isn't always more efficient. The overhead of coordinating between them grows significantly and the network requirements are becoming too large for the benefits**





Memory question

- **All previous examples assume that we can fit at least one input sample forward pass into our GPU memory. This is not always the case! Larger models don't fit into a single GPU**



ZeRO (Zero Redundancy Optimizer)

- **DP is an efficient way to scale training, the naive replication of optimizer states, gradients, and parameters across each DP rank introduces a significant memory redundancy**

ZeRO (Zero Redundancy Optimizer)

- **DP** is an efficient way to scale training, the naive replication of optimizer states, gradients, and parameters across each DP rank introduces a significant memory redundancy
- **Idea:** eliminate memory redundancy by partitioning the optimizer states, gradients, and parameters across the data parallel dimension, while still allowing computation with the full set of parameters

ZeRO (Zero Redundancy Optimizer)

- **DP is an efficient way to scale training, the naive replication of optimizer states, gradients, and parameters across each DP rank introduces a significant memory redundancy**

- **Idea: eliminate memory redundancy by partitioning the optimizer states, gradients, and parameters across the data parallel dimension, while still allowing computation with the full set of parameters**

  > **ZeRO-1:** optimizer state partitioning

  > **ZeRO-2:** optimizer state + gradient partitioning

  > **ZeRO-3** (also called FSDP for “Fully-Sharded Data Parallelism”): optimizer state + gradient + parameter partitioning

ZeRO (Zero Redundancy Optimizer)

- **DP is an efficient way to scale training, the naive replication of optimizer states, gradients, and parameters across each DP rank introduces a significant memory redundancy**

- **Idea: eliminate memory redundancy by partitioning the optimizer states, gradients, and parameters across the data parallel dimension, while still allowing computation with the full set of parameters**

  > **ZeRO-1:** optimizer state partitioning

  > **ZeRO-2:** optimizer state + gradient partitioning

  > **ZeRO-3** (also called FSDP for “Fully-Sharded Data Parallelism”): optimizer state + gradient + parameter partitioning

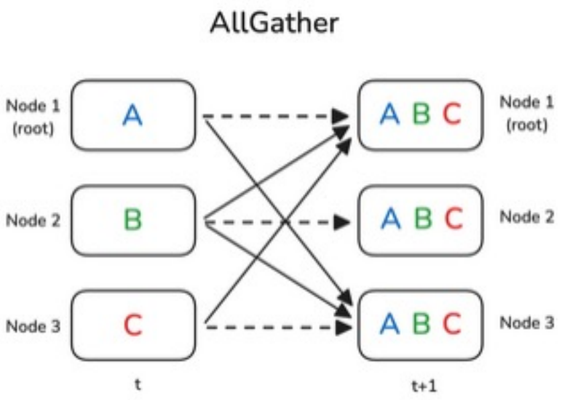
- **ZeRO is to shard os/g/p across the DP ranks, each node only storing a slice of the items which are reconstructed when and if needed**

ZeRO-1

- The optimizer states are partitioned into N_d equal parts where N_d is the DP degree. Each model replica distributed on each DP rank only keeps track of $\frac{1}{N_d}$ of the optimizer states

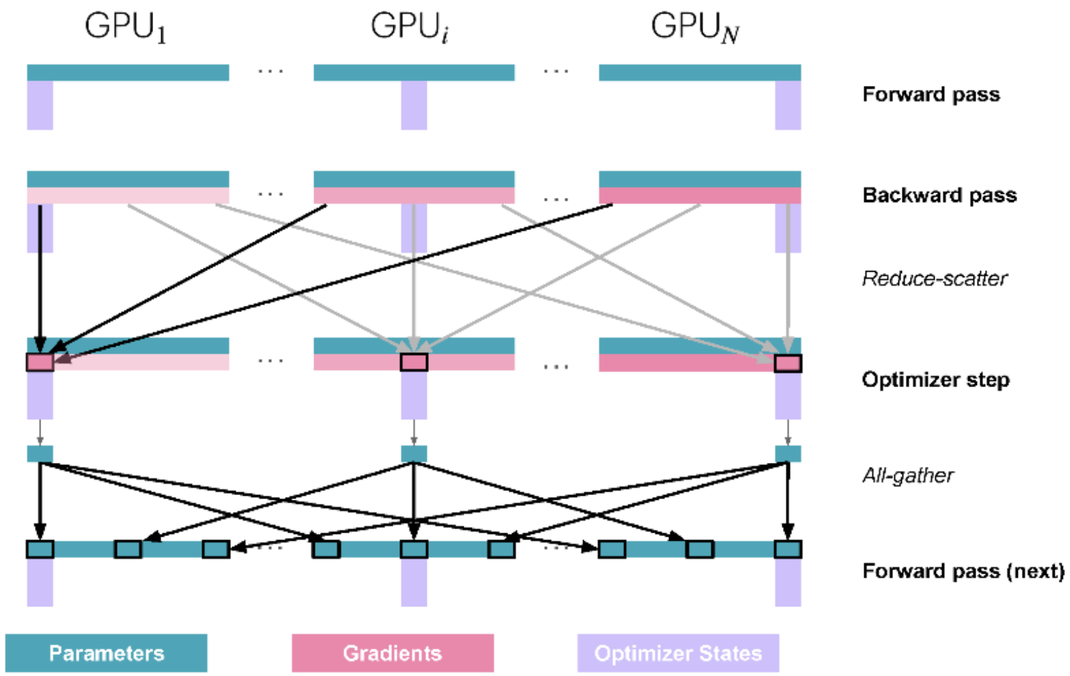
ZeRO-1

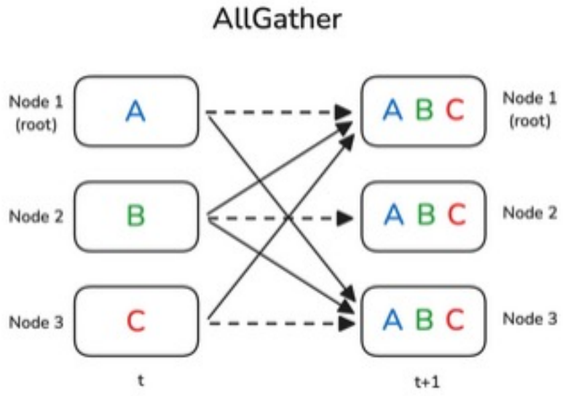
- The optimizer states are partitioned into N_d equal parts where N_d is the DP degree. Each model replica distributed on each DP rank only keeps track of $\frac{1}{N_d}$ of the optimizer states
- During the forward pass, each replica need all the parameters, we thus need to add an additional all-gather

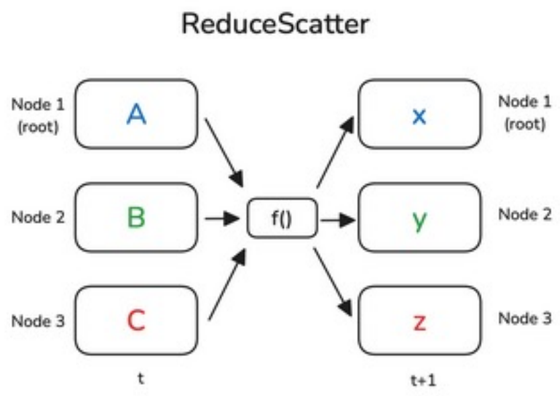


ZeRO-1

- The optimizer states are partitioned into N_d equal parts where N_d is the DP degree. Each model replica distributed on each DP rank only keeps track of $\frac{1}{N_d}$ of the optimizer states
- During the forward pass, each replica need all the parameters, we thus need to add an additional all-gather





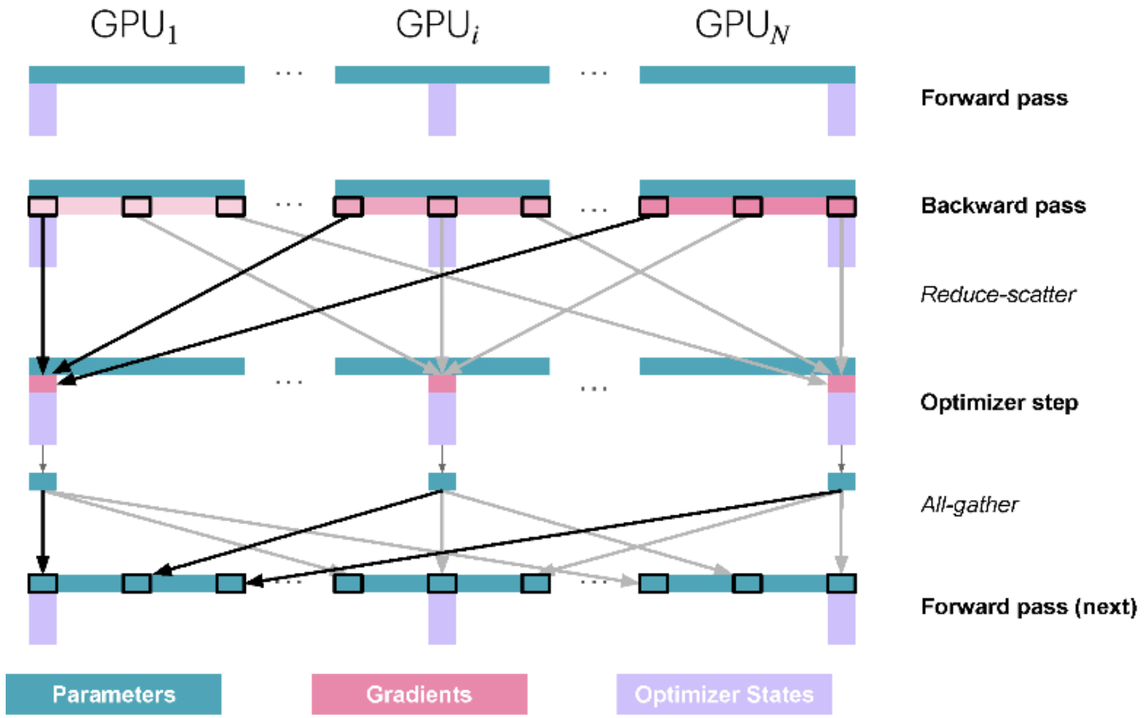


ZeRO-2

- Let's shard gradient as well similarly to the optimizer states. Where we only spread the $\frac{1}{N_d}$ gradients needed in memory, thus saving more memory compared to ZeRO-1

ZeRO-2

- **Let's shard gradient as well similarly to the optimizer states. Where we only spread the $\frac{1}{N_d}$ gradients needed in memory, thus saving more memory compared to ZeRO-1**



ZeRO-3 a.ka. FSDP

- **Extend the ZeRO-2 approach of sharding optimizer states and gradients over DP replicas up to sharding the model's parameters.**

ZeRO-3 a.ka. FSDP

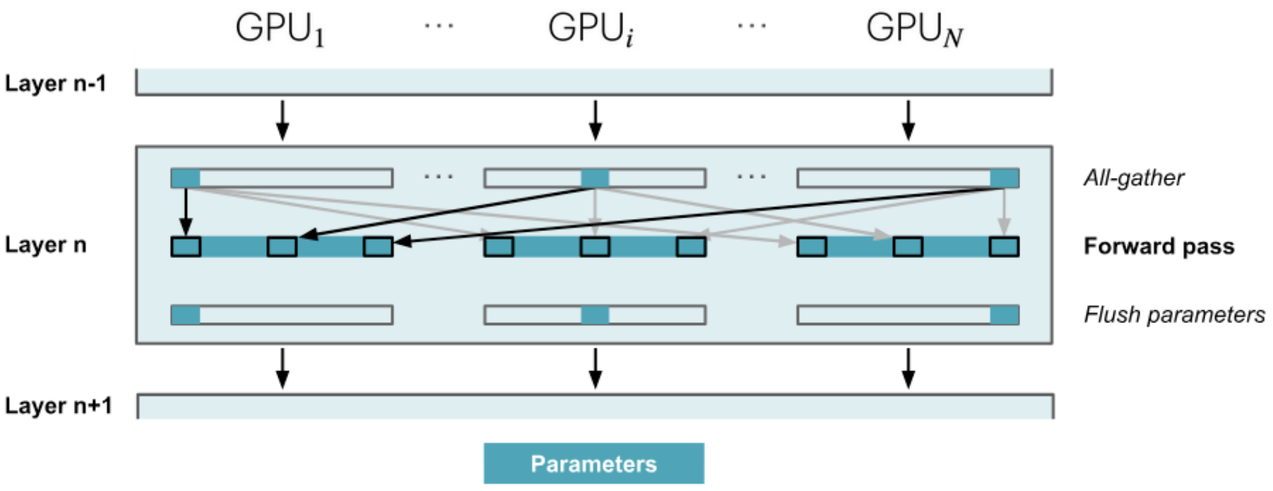
- **Extend the ZeRO-2 approach of sharding optimizer states and gradients over DP replicas up to sharding the model's parameters.**

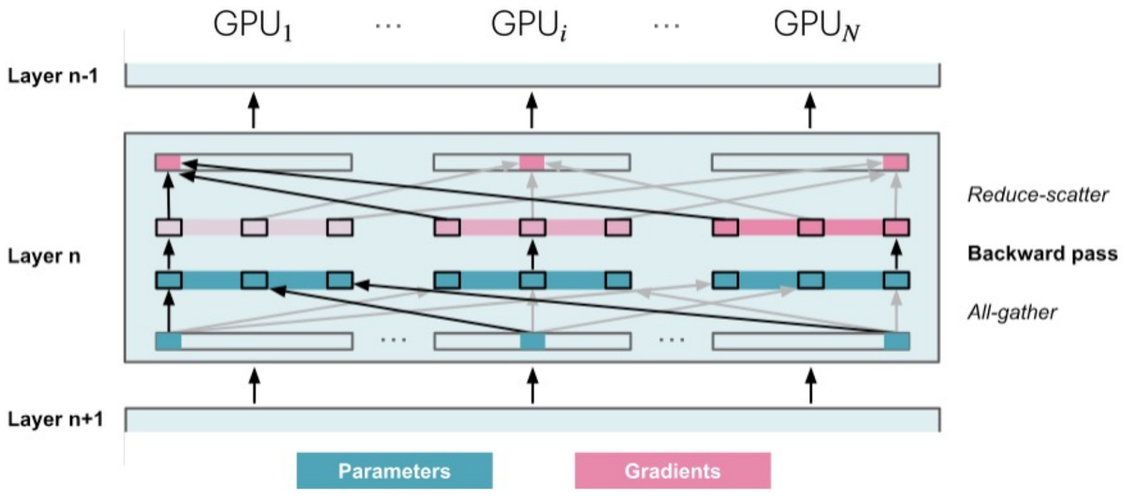
- **To perform forward/backward pass we gather parts on-demand when we need them**

ZeRO-3 a.ka. FSDP

- **Extend the ZeRO-2 approach of sharding optimizer states and gradients over DP replicas up to sharding the model's parameters.**

- **To perform forward/backward pass we gather parts on-demand when we need them**





Recap

- Single-GPU fundamentals: what occupies memory, gradient checkpointing, gradient accumulation
- Mixed-precision training (BF16/FP32)
- Data Parallelism replicates the model across GPUs, processing different micro-batches in parallel and synchronizing gradients via all-reduce
- ZeRO (stages 1→2→3) progressively shards optimizer states, gradients, and parameters across the DP ranks, dramatically cutting per-GPU memory with minimal communication overhead

Next:

- What if a layer of the model not fits in a single GPU? TP, PP, CP